Lab Assignment 5.5

Name: P. Sanjay Kumar
Roll No: 2403A52L06
Batch -50

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week3** –	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number: 5.5(Present assignment number)/**24**(Total number of assignments)			

Q.No.	**Question**	***Expected Time to complete***

1	**Lab 5: Ethical Foundations – Responsible AI Coding Practices** **Lab Objectives:** • To explore the ethical risks associated with AI-generated code. • To recognize issues related to security, bias, transparency, and copyright. • To reflect on the responsibilities of developers when using AI tools in software development. • To promote awareness of best practices for responsible and ethical AI coding. **Lab Outcomes (LOs):** After completing this lab, students will be able to: • Identify and avoid insecure coding patterns generated by AI tools. • Detect and analyze potential bias or discriminatory logic in AI-generated outputs. • Evaluate originality and licensing concerns in reused AI-generated code. • Understand the importance of explainability and transparency in AI-assisted programming. • Reflect on accountability and the human role in ethical AI coding practices. **Task Description #1 (Transparency in Algorithm Optimization)** **Task:** Use AI to generate two solutions for checking prime numbers: • Naive approach(basic) • Optimized approach **Expected Output:** • Code for both methods. • Transparent explanation of time complexity. • Comparison highlighting efficiency improvements.	Week3 -

Prompt:

Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Code:

lab assignment5.5.py > is_prime_optimized

```python
import math
# Naive method
def is_prime_basic(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
# Optimized method
def is_prime_optimized(n):
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    for i in range(3, int(math.sqrt(n)) + 1, 2):
        if n % i == 0:
            return False
    return True
# User input
num = int(input("Enter a number: "))
print("Basic method result:", is_prime_basic(num))
print("Optimized method result:", is_prime_optimized(num))
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\sanjay\AI Assistant coding> & C:\Users\
oding/lab assignment5.5.py"
Enter a number: 3
Basic method result: True
Optimized method result: True
PS C:\Users\sanjay\AI Assistant coding> & C:\Users\
oding/lab assignment5.5.py"
Enter a number: 10
Basic method result: False
Optimized method result: False
PS C:\Users\sanjay\AI Assistant coding>
```

Explanation:

The naive approach code checks whether the given number is prime

	or non prime number. Time Complexity: O(n) — slow for large numbers. The optimized version code checks skips even numbers and stops checking after the square root of the number. Time Complexity: O(√n) — much faster.	
	Task Description #2 (Transparency in Recursive Algorithms) **Objective:** Use AI to generate a recursive function to calculate Fibonacci numbers. **Instructions:** 1. Ask AI to add clear comments explaining recursion. 2. Ask AI to explain base cases and recursive calls. **Expected Output:** • Well-commented recursive code. • Clear explanation of how recursion works. • Verification that explanation matches actual execution. **Prompt:** Generate a python code to generate a recursive function to calculate the Fibonacci numbers. Code:	

```python
#task2
def fibonacci(n):
    # Base case 1:
    # If n is 0, Fibonacci value is 0
    if n == 0:
        return 0

    # Base case 2:
    # If n is 1, Fibonacci value is 1
    if n == 1:
        return 1

    # Recursive case:
    # Sum of previous two Fibonacci numbers
    return fibonacci(n - 1) + fibonacci(n - 2)


# Take input from the user
n = int(input("Enter a number: "))

# Call the recursive function and print the result
print("Fibonacci number is:", fibonacci(n))
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\sanjay\AI Assistant coding> & C:\Users\M
oding/lab assignment5.5.py"
Enter a number: 10
Fibonacci number is: 55
PS C:\Users\sanjay\AI Assistant coding>
```

Explanation:

In base cases 1 and 2 we have given value 0 and 1 so it stops the recursive calls and in the recursive case the function calls itself to find the fibonacci values of the two previous numbers.

Task Description #3 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and

processes data.

Prompt:

"Generate code with proper error handling and clear explanations for each exception."

Expected Output:

- Code with meaningful exception handling.
- Clear comments explaining each error scenario.
- Validation that explanations align with runtime behavior.

Prompt:

"Generate code with proper error handling and clear explanations for each exception."

Code:

```python
#task3
def calculate_average(filename):
    try:
        # Step 1: Try to open the file
        with open(filename, "r") as file:
            lines = file.readlines()
        # Step 2: Check if file is empty
        if not lines:
            raise ValueError("File is empty. No data to process.")

        numbers = []

        # Step 3: Convert each line into an integer
        for line in lines:
            numbers.append(int(line.strip()))

        # Step 4: Calculate average
        average = sum(numbers) / len(numbers)
        print("Average of numbers:", average)

    except FileNotFoundError:
        # Triggered when file does not exist
        print("Error: File not found. Please check the file name or path.")

    except ValueError as ve:
        # Triggered for:
        # 1) Non-numeric data
        # 2) Empty file (manually raised)
        print("Error:", ve)

    except ZeroDivisionError:
        # Triggered if division by zero occurs
        print("Error: Cannot divide by zero. No valid numbers found.")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\sanjay\AI Assistant coding> & C:\Users
oding/lab assignment5.5.py"
Enter file name: test.txt
Error: File not found. Please check the file name
Program execution completed.
PS C:\Users\sanjay\AI Assistant coding>
```

Explanation:

If the file is empty or there is no file with the name we have given,it clearly shows that file not found. If there is the correct file uploaded we will get the average in all cases**.**

Task Description #4 (Security in User Authentication)

Task: Use an AI tool to generate a Python-based login system.

Analyze: Check whether the AI uses secure password handling practices.

Expected Output:

- Identification of security flaws (plain-text passwords, weak validation).
- Revised version using password hashing and input validation.
- Short note on best practices for secure authentication.

Prompt:

Generate a code in python based login system and check whether that the Ai uses secure password handling.

Code:

```python
#task4
import hashlib
import getpass
import re

# Stores username : hashed_password
user_db = {}

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def is_strong_password(password):
    """
    Password rules:
    - Minimum 8 characters
    - At least 1 uppercase letter
    - At least 1 lowercase letter
    - At least 1 digit
    - At least 1 special character
    """
    if len(password) < 8:
        return False
    if not re.search(r"[A-Z]", password):
        return False
    if not re.search(r"[a-z]", password):
        return False
    if not re.search(r"\d", password):
        return False
    if not re.search(r"[!@#$%^&*]", password):
        return False
    return True

def register():
    username = input("Enter new username: ")

    if username in user_db:
        print("Username already exists. Try login.")
        return

    password = getpass.getpass("Enter password: ")

    if not is_strong_password(password):
        print("Weak password! Use upper, lower, digit, special & 8+ chars.")
        return

    confirm = getpass.getpass("Confirm password: ")

    if password != confirm:
        print("Passwords do not match!")
        return

    user_db[username] = hash_password(password)
    print(f"User '{username}' registered successfully!")

def login():
    username = input("Enter username: ")

    if username not in user_db:
        print("User not found. Register first.")
        return

    password = getpass.getpass("Enter password: ")

    if user_db[username] == hash_password(password):
        print(f"Welcome, {username}! Login successful.")
    else:
        print("Incorrect password. Try again.")

# Main menu loop
while True:
    choice = input("\n[register/login/exit]: ").lower()

    if choice == "register":
        register()
    elif choice == "login":
        login()
    elif choice == "exit":
        print("Exiting program...")
        break
    else:
        print("Invalid option.")

```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

[register/login/exit]: register
Enter new username: sanjay
Enter password:
Confirm password:
User 'sanjay' registered successfully!

[register/login/exit]: login
Enter username: sanjay
Enter password:
Welcome, sanjay! Login successful.

[register/login/exit]:
```

Explanation:

When registering, the program checks if the password is strong and hides input; it shows success only if all checks pass. During login, it verifies the hashed password and welcomes the user if correct, or shows an error if the username or password is wrong.

Task Description #5 (Privacy in Data Logging)

Task: Use an AI tool to generate a Python script that logs user activity (username, IP address, timestamp).

Analyze: Examine whether sensitive data is logged unnecessarily or insecurely.

Expected Output:

- Identified privacy risks in logging.
- Improved version with minimal, anonymized, or masked logging.
- Explanation of privacy-aware logging principles.

Prompt:

Generate a code in python script that logs user activity (username, IP address, timestamp).Examine whether sensitive data is logged unnecessarily or insecurely.

Code:

```python
#task5
import logging
import socket
import hashlib
from datetime import datetime

# Configure logging
logging.basicConfig(
    filename='user_activity_safe.log',
    level=logging.INFO,
    format='%(asctime)s - %(user_id)s - %(ip_masked)s - %(action)s'
)

def get_ip_masked():
    try:
        hostname = socket.gethostname()
        ip = socket.gethostbyname(hostname)

        # Mask last octet for privacy (IPv4)
        parts = ip.split('.')
        if len(parts) == 4:
            parts[-1] = 'xxx'
            return '.'.join(parts)

        return "Unknown IP"

    except socket.error:
        return "Unknown IP"

def anonymize_username(username):
    # Hash username and keep only first 8 chars
    return hashlib.sha256(username.encode()).hexdigest()[:8]
```

Output:

```
PS C:\Users\sanjay\AI Assistant coding> & C:\Users\Naveen\AppData\L
oding/lab assignment5.5.py"
Enter username: sanjay
Enter action performed: sanju@123
Activity logged: user=021fb044, ip=192.168.7.xxx, action=sanju@123
PS C:\Users\sanjay\AI Assistant coding>
```

Explanation:

The code logs user activity safely by asking usernames and masking IP addresses, avoiding any direct personal identification. It also fixes a logging error by using a custom key action instead of message.